

Data types and typecasting

and why they all matter

3.1: What are data types?

Back in Day 1, we said that a variable stores *data*. But not all data is the same! The text `"Tony"` and the number `25` are fundamentally different kinds of information, and Python treats them differently. The *type* of a piece of data is Python’s way of answering the question: “What kind of thing is this, and what am I allowed to do with it?”

Every single value in Python has a type. That type determines what operations make sense. You can multiply two numbers, but multiplying two names makes no sense. You can capitalize a piece of text, but capitalizing the number `7` is nonsense. Python keeps track of the type of every value precisely so it knows which operations are allowed and which ones should raise an error.

For now, we will focus on three of the most fundamental types. There is a fourth basic type called `bool` (for true/false values), but we will save that one for a later day.

Type	Name	Example Values	What It Represents
<code>str</code>	String	<code>"Tony"</code> , <code>"Howdy!"</code> , <code>"123"</code>	Text
<code>int</code>	Integer	<code>25</code> , <code>0</code> , <code>-7</code>	Whole numbers
<code>float</code>	Floating-point	<code>3.14</code> , <code>-0.5</code> , <code>2.0</code>	Numbers with decimals

3.1.1: Checking a value’s type with `type()`

Python gives you a built-in function called `type()` that tells you the type of any value. This is incredibly handy when you are not sure what you are working with:

```
print(type("Tony"))
print(type(25))
print(type(3.14))
```

```
<class 'str'>
<class 'int'>
<class 'float'>
```

Notice the output says `<class 'str'>`, `<class 'int'>`, and `<class 'float'>`. Don’t worry about the word “class” for now since the important part is the name inside the quotes, which tells you the type.

3.1.2: Strings (`str`)

A *string* is Python's way of storing text. Any time you want to store words, sentences, names, or really any sequence of characters, you use a string. You create one by wrapping the text in quotes:

```
greeting = "Howdy, world!"
print(greeting)
print(type(greeting))
```

```
Howdy, world!
<class 'str'>
```

You can use either double quotes (`"..."`) or single quotes (`'...'`) for strings since Python treats them the same way. Just be consistent, and make sure the opening quote matches the closing quote.

Watch out — Quotes are what make a string a string!

This is one of the most important ideas in this whole chapter. The *quotes* are what tell Python “treat this as text.” Look carefully at the difference between these two:

```
print(type(123))
print(type("123"))
```

```
<class 'int'>
<class 'str'>
```

`123` is the *number* one hundred twenty-three. `"123"` is a *string* that happens to contain three digit characters. They look similar to us, but to Python they are completely different types with completely different rules!

Because a string is just text, math operators behave very differently on strings than you might expect. The `+` operator does not add but rather *concatenates* (glues text together):

```
first = "Howdy"
second = "world"
print(first + " " + second)
```

```
Howdy world
```

And `*` with a string and a number repeats the text:

```
print("ha" * 3)
```

```
hahaha
```

3.1.3: Integers (`int`)

An *integer* is a whole number: no decimal point, no fractional part. Integers can be positive, negative, or zero:

```
age = 25
temperature = -7
count = 0

print(type(age))
```

```
<class 'int'>
```

Integers are what you reach for when you are counting things or working with values that are inherently whole: the number of students in a class, someone's age in years, the number of times a loop should run, and so on. As we saw on Day 2, math on integers behaves exactly the way you learned in school, with the one wrinkle being the two kinds of division (`/` versus `//`).

3.1.4: Floating-point numbers (`float`)

A *float* (short for “floating-point number”) is a number that has a decimal point. Floats are what you use whenever you need fractional precision:

```
pi = 3.14159
half = 0.5
negative = -2.75

print(type(pi))
```

```
<class 'float'>
```

Here is a subtle but very important point: a number is a `float` the moment it has a decimal point, *even if the fractional part is zero*.

```
print(type(5))
print(type(5.0))
```

```
<class 'int'>
<class 'float'>
```

`5` is an `int`, but `5.0` is a `float`, even though they represent the same mathematical value. This matters more than you might think, because certain operations produce floats automatically. Recall from Day 2 that regular division `/` *always* produces a float, even when the numbers divide evenly:

```
print(10 / 2)
print(type(10 / 2))
```

```
5.0
<class 'float'>
```

The result is `5.0` (a float), not `5` (an int), because `/` always hands back a float.

Note — Floats are not always perfectly precise

Because of how computers store decimal numbers in binary, some floats end up ever so slightly off from what you would expect (for example, `0.1 + 0.2` gives `0.30000000000000004`). This is not a bug in your code: it is a fundamental limitation of representing decimals in binary. You don't need to worry about it right now, but file it away for later.

3.2: Typecasting

So far each value has arrived with a fixed type. But very often you will have a value of one type and need it as another. Maybe you have the string `"25"` (text) but you need the number `25` so you can do math with it. The process of converting a value from one type to another is called *typecasting* (or just *casting*).

Python gives you three conversion functions, one for each type. You wrap the value you want to convert inside the function:

Function	Converts a value into a...
<code>str()</code>	string
<code>int()</code>	integer
<code>float()</code>	float

Watch out — Casting produces a new value!

When you cast a value, Python hands you back a brand-new value of the new type. The original value is untouched. If you want to keep the converted result, you must store it in a variable:

```
number = 25
text = str(number)    # `text` now holds "25", but `number` is still the int 25
```

3.2.1: Casting to a string with `str()`

`str()` is the most forgiving of the three functions: it can turn *any* value into its text representation. This never fails, because every value has some way of being displayed as text.

```
print(str(25))
print(str(3.14))
print(str(-7))
```

25

3.14

-7

The output looks the same as the original number, but the *type* is now a string. This becomes obvious the moment you try to do “math” on the result: remember that `+` glues strings together (concatenation):

```
print(str(25) + str(25))
```

2525

Instead of `50`, we get `"2525"`, because we glued the string `"25"` onto the string `"25"`.

`str()` is most commonly used when you want to combine a number with some text for display, since Python will not let you glue a number directly onto a string:

```
age = 25
print("I am " + age + " years old")
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[132], line 2
      1 age = 25
----> 2 print("I am " + age + " years old")
```

TypeError: can only concatenate str (not "int") to str

The fix is to cast the number to a string first:

```
age = 25
print("I am " + str(age) + " years old")
```

I am 25 years old

3.2.2: Casting to an integer with `int()`

`int()` is much pickier than `str()`, and understanding *exactly* what it will and won't accept is one of the most common early stumbling blocks. Let's go through the rules carefully. When you cast a float to an integer, Python simply *throws away* everything after the decimal point. Formally, this is known as **truncation**.

```
print(int(3.9))
print(int(3.2))
print(int(-3.9))
```

3
3
-3

⚠ Watch out — `int()` does not round!

This trips up nearly everyone at least once. If you expected `int(3.9)` to give `4`, you will get bugs. If you actually want rounding, Python has a separate `round()` function for that:

```
print(int(3.9))    # truncates to 3
print(round(3.9))  # rounds to 4
```

```
3
4
```

3.2.3: `int()` on a string

`int()` can convert a string into an integer, but *only* if that string contains a clean, whole number and nothing else. This works:

```
print(int("25"))
print(int("-7"))
```

```
25
-7
```

But all of the following will crash your program, because the string does not represent a plain whole number:

```
print(int("3.14"))
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[137], line 1
----> 1 print(int("3.14"))
```

ValueError: invalid literal for int() with base 10: '3.14'

```
print(int("hello"))
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[138], line 1
----> 1 print(int("hello"))
```

ValueError: invalid literal for int() with base 10: 'hello'

```
print(int("25 dollars"))
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[139], line 1
----> 1 print(int("25 dollars"))
```

ValueError: invalid literal for int() with base 10: '25 dollars'

The rules for casting a string to an integer are strict:

- The string must contain only digits (an optional leading `+` or `-` is allowed).
- **No decimal point is allowed**, even if the part after it is zero. `int("5.0")` fails!
- No letters, symbols, currency signs, or other stray characters.
- Surrounding spaces (`" 25 "`) are actually tolerated and stripped away, but spaces *inside* the number are not.

Note — “But I have a string like `"3.14"` and I need an integer!”

You cannot go straight from `"3.14"` to an int, because `int()` refuses strings with decimal points. The trick is to convert in two steps: first to a float, then to an int.

```
print(int(float("3.14")))
```

3

`float("3.14")` gives the float `3.14`, and then `int(...)` truncates it down to `3`.

3.2.4: A quick summary of `int()`

You give <code>int()</code> ...	Result
A float like <code>3.9</code>	Truncates toward zero → <code>3</code>
A clean whole-number string like <code>"25"</code>	The integer <code>25</code>
A decimal string like <code>"3.14"</code>	Error!
A non-numeric string like <code>"hello"</code>	Error!

3.2.5: Casting to a float with `float()`

`float()` sits comfortably in between the other two: more flexible than `int()`, but still requires numeric input when given a string.

Casting an integer to a float simply adds a `.0` on the end:

```
print(float(25))
print(float(-7))
```

25.0
-7.0

Unlike `int()` with strings, `float()` happily accepts strings that contain a decimal point, as well as strings that are whole numbers:

```
print(float("3.14"))
print(float("25"))
print(float("-0.5"))
```

3.14
25.0
-0.5

But just like `int()`, it will refuse anything that isn't a valid number:

```
print(float("hello"))
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[143], line 1
----> 1 print(float("hello"))
```

ValueError: could not convert string to float: 'hello'

 **Note — Why does `float()` accept "3.14" but `int()` rejects it?**

Think about what each type *is*. A float is allowed to have a decimal part, so a string with a decimal point maps onto it perfectly. An integer, by definition, has no decimal part, so Python refuses to guess what you meant by the `.14`. Should it chop it? Round it? Rather than make that decision for you, `int()` simply says no. This is why the two-step `int(float("3.14"))` trick from earlier is necessary: you first make the decision to move to a float, then explicitly choose to truncate.

3.2.6: Putting it all together

Here is the big-picture map of what converts cleanly into what:

From (row) / To (column)	str (with <code>str()</code>)	int (with <code>int()</code>)	float (with <code>float()</code>)
int (e.g. 25)	"25"	25	25.0
float (e.g. 3.9)	"3.9"	truncates to 3	3.9
str (e.g. "25")	"25"	only if whole number	only if numeric

The takeaways to burn into memory:

- 1. **Everything can become a string:** `str()` never fails.

2. `int()` truncates floats.
3. `int()` rejects strings with decimal points. Go through `float()` first if needed.
4. `float()` accepts numeric strings with or without a decimal point.

3.3: Why casting matters so much

The single most common place you will use casting is with *user input*. When you ask a user to type something in (which you will learn how to do soon), Python always hands you back a **string**, even if the user typed a number. If you forget to cast it, your “math” will do string operations instead:

```
user_typed = "10"      # imagine the user typed 10
print(user_typed + "5") # string glue: "105"
print(int(user_typed) + 5) # real math: 15
```

105

15

Getting comfortable with types and casting now will save you an enormous number of confusing bugs down the road. When something behaves strangely, one of the very first things to check is: *what type is this value, really?* And `type()` is right there to tell you.